SOFTWARE METRICS

chapter Seven

COCOMO Model - Software Engineering

The Constructive Cost Model (COCOMO) It was proposed by Barry Boehm in 1981 and is based on the study of 63 projects, which makes it one of the best-documented models.
It is a Software Cost Estimation Model that helps predict the effort, cost, and schedule required for a software development project.

SOFTWARE METRICS

chapter Seven

What is the COCOMO Model?

COCOMO Model is a procedural cost estimate model for Software Projects and is often used as a process of reliably predicting the various parameters associated with making a project such as size, effort, cost, time, and quality. The key parameters that define the quality of any Software Product, which are also an outcome of COCOMO, are primarily effort and schedule.

SOFTWARE METRICS

chapter Seven

what is COCOMO Model

Types of Projects in COCOMO Model

In the COCOMO model, software projects are categorized into three types based on their complexity, size, and the development environment. These types are:

A. organic : A software project is said to be an organic type if the team size required is adequately small, the problem is well understood and has been solved in the past and also the team members have a nominal experience regarding the problem.

SOFTWARE METRICS

chapter Seven

COCOMO Model - Software Engineering

B. Semi-detached

A software project is said to be a Semi-detached type if the vital characteristics such as team size, experience, and knowledge of the various programming environments lie in between organic and embedded.

C. Embedded

A software project requiring the highest level of complexity, creativity, and experience requirement falls under this category. Such software requires a larger team size than the other two models and also the developers need to be sufficiently experienced and creative to develop such complex models.

SOFTWARE METRICS

chapter Seven

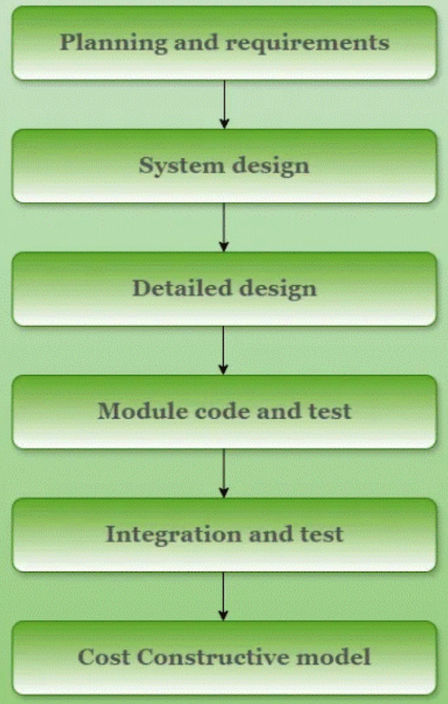
Structure of COCOMO Model

Detailed COCOMO incorporates all characteristics of the intermediate version with an assessment of the cost driver's impact on each step of the Software Engineering Process.
In detailed COCOMO, the whole software is divided into different modules and then we apply COCOMO in different modules to estimate effort and then sum the effort.
The Six phases of detailed COCOMO are:

SOFTWARE METRICS

chapter Seven

Six phases of COCOMO Model



SOFTWARE METRICS

chapter Seven

Phases of COCOMO Model

1. Planning and requirements: This initial phase involves defining the scope, objectives, and constraints of the project. It includes developing a project plan that outlines the schedule, resources, and milestones
2. System design: : In this phase, the high-level architecture of the software system is created. This includes defining the system's overall structure, including major components, their interactions, and the data flow between them.

SOFTWARE METRICS

chapter Seven

Phases of COCOMO Model

3. Detailed design: This phase involves creating detailed specifications for each component of the system. It breaks down the system design into detailed descriptions of each module, including data structures, algorithms, and interfaces.

4.Module code and test: This involves writing the actual source code for each module or component as defined in the detailed design. It includes coding the functionalities, implementing algorithms, and developing interfaces.

SOFTWARE METRICS

chapter Seven

Phases of COCOMO Model

5.Integration and test: This phase involves combining individual modules into a complete system and ensuring that they work together as intended.

6.Cost Constructive model: The Constructive Cost Model (COCOMO) is a widely used method for estimating the cost and effort required for software development projects.

SOFTWARE METRICS

chapter Seven

Types of COCOMO Model

1. Basic COCOMO Model

The Basic COCOMO model is a straightforward way to estimate the effort needed for a software development project. It uses a simple mathematical formula to predict how many person-months of work are required based on the size of the project, measured in thousands of lines of code (KLOC).

$E = a*(KLOC)_b$ PM

$Tdev = c*(E)_d$

Person required = Effort/ Time

Where,

E is effort applied in Person-Months

KLOC is the estimated size of the software product indicate in Kilo Lines of Code

Tdev is the development time in months

SOFTWARE METRICS

chapter Seven

Types of COCOMO Model

Suppose that a Basic project was estimated to be 400 KLOC (kilo lines of code). Calculate effort and time for each of the three modes of development. a, b, c are constants determined by the category of software project.

Software Projects

abcd

Organic

2.4=a

1.05=b

2.5=c

0.38=d

Semi-Detached

3.0=a, 0.35=d

12=b, 5=c

SOFTWARE METRICS

chapter Seven

Types of COCOMO Model

1. For organic mode,
   - effort = $2.4 \times (400)_{1.05} \approx 1295$ person-month.
   - dev. time = $2.5 \times (1295)_{0.38} \approx 38$ months.
2. For semi-detach mode,
   - effort = $3 \times (400)_{1.12} \approx 2462$ person-month.
   - dev. time = $2.5 \times (2462)_{0.35} \approx 38$ months.

SOFTWARE METRICS

chapter Seven

Types of COCOMO Model

2. Intermediate COCOMO Model

The basic COCOMO model assumes that the effort is only a function of the number of lines of code and some constants evaluated according to the different software systems. However, in reality, no system's effort and schedule can be solely calculated based on Lines of Code. For that, various other factors such as reliability, experience, and Capability. These factors are known as Cost Drivers (multipliers) and the Intermediate Model utilizes 15 such drivers for cost estimation.

SOFTWARE METRICS

chapter Seven

Types of COCOMO Model

Classification of Cost Drivers and their Attributes:
The cost drivers are divided into four categories
Product attributes:

- Required Software Reliability extent
- Size of the application database
- The complexity of the product

SOFTWARE METRICS

chapter Seven

Types of COCOMO Model

Hardware attributes:

- Run-time performance constraints
- Memory constraints
- The volatility of the virtual machine environment
- Required turnabout time

SOFTWARE METRICS

chapter Seven

Types of COCOMO Model

Personal attributes:

- Analyst capability
- Software engineering capability
- Application experience
- Virtual machine experience
- Programming language experience

SOFTWARE METRICS

chapter Seven

Types of COCOMO Model

Project attributes:

- Use of Software Tools.
- Application of Software Engineering Methods.
- Required development schedule.

The Effort Adjustment Factor (EAF) is determined by multiplying the effort multipliers associated with each of the 15 attributes.

SOFTWARE METRICS

chapter Seven

Types of COCOMO Model

$E = a*(KLOC)_b * EAF$ PM

$Tdev = c*(E)_d$

Where,

- E is effort applied in Person-Months
- KLOC is the estimated size of the software product indicate in Kilo Lines of Code
- EAF is the Effort Adjustment Factor (EAF) is a multiplier used to refine the effort estimate obtained from the basic COCOMO model.
- Tdev is the development time in months
- a, b, c are constants determined by the category of software project given in below table.

SOFTWARE METRICS

chapter Seven

3. Detailed COCOMO Model

Detailed COCOMO goes beyond Basic and Intermediate COCOMO by diving deeper into project-specific factors. It considers a wider range of parameters, like team experience, development practices, and software complexity.

1. NASA Space Shuttle Software Development: NASA estimated the time and money needed to build the software for the Space Shuttle program using the COCOMO model. NASA was able to make well-informed decisions on resource allocation and project scheduling by taking into account variables including project size, complexity, and team experience.

SOFTWARE METRICS

chapter Seven

Phases of COCOMO Model

COCOMO ICOCOMO II

COCOMO I is useful in the waterfall models of the software development cycle.

it provides estimates of effort and schedule.

This model is based upon the linear reuse formula.

Effort equation's exponent is determined by 3 development modes.

This model is also based upon the assumption of reasonably stable requirements.

COCOMO II is useful in non-sequential, rapid development and reuse models of software.

It provides estimates that represent one standard deviation around the most likely estimate.

This model is based upon the non linear reuse formula

This model is also based upon reuse model which looks at effort needed to understand and estimate.

Effort equation's exponent is determined by 5 scale factors.

SOFTWARE METRICS

chapter Seven

Advantages of SLIM Cost Model

- Ideal for Large Projects: SLIM is highly effective and more accurate than other models for large-scale, complex software projects, especially those exceeding 70,000 lines of code.
- Fewer Parameters Needed: Compared to models like COCOMO '81 or COCOMO II, SLIM requires fewer parameters to generate an estimate.
- Incorporates Time-Effort Tradeoff: It properly considers the dynamic relationship between development time, effort, and cost, recognizing that adding staff does not always speed up development linearly.
- Top-Down Estimation: SLIM is effective at the early stages of a project's life cycle when only high-level requirements are known.

SOFTWARE METRICS

chapter Seven

Drawbacks of SLIM Cost Model

Drawbacks of SLIM Cost Model

- Extremely Sensitive to Technology Factor: The model's accuracy is heavily dependent on the "technology constant," which represents the team's productivity. Small miscalculations in this factor can lead to large inaccuracies in effort estimation.
- Unsuitable for Small Projects: SLIM is generally not recommended for small projects (small LOC), as the Rayleigh distribution assumes a significant development cycle that does not apply to short efforts.
- Dependence on Historical Data: To be effective, SLIM requires accurate historical data to determine the productivity index. If a company has not previously collected this data, the estimate will not be accurate.

SOFTWARE METRICS

chapter Seven

Constraint model

Constraint is something that limits or restricts something else

• A constraint is not a bias or a preference. Implementing a rule such as “games should be scheduled on Friday nights if possible” is not a constraint, it’s a preference. A rule such as “customer ABC’s orders should be processed out of DC 123” is also not a binding constraint if viable alternatives are allowed to be considered.

• Some industry pundits believe that using preferences and biases in the form of heuristic rules, in lieu of hard constraints, is perfectly acceptable — but is it? What it comes down to is how important profitability, predictability, agility, and accountability are to the company’s management and their investors.

SOFTWARE METRICS

chapter Seven

SLIM Cost Model

The Putnam model is an empirical software effort estimation model. The original paper by Lawrence H. Putnam published in 1978 is seen as pioneering work in the field of software process modelling. As a group, empirical models work by collecting software project data (for example, effort and size) and fitting a curve to the data. Future effort estimates are made by providing size and calculating the associated effort using the equation which fit the original data (usually with some error).